

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería de Software

Prueba Práctica – Unidad IV

Ingeniería de Requisitos (ISR-401)

Caso: Sistema de Gestión de Pedidos

Docente: Ing. Gleiston Cicerón Guerrero Ulloa, PhD

Estudiante: Mesias Quijije Jhon Alexander

Asignatura: Ingeniería de Requisitos – ISR-401

Repositorio GitHub:

<https://github.com/jhonaelx24/Evaluaci-n-Pr-ctica-de-la-Unidad-IV>

**Resumen de cuestionario**
QUERRERO ULLOA GLEISTON CICERON
SOFT-R INGENIERÍA DE REQUERIMIENTOS Nivel 4TO NIVEL Paralelo A

EVALUACIÓN SUMATIVA UNIDAD IV

Intentos permitidos: 1
Este cuestionario se cerró el Martes, 11 de Agosto de 2026, 12:45
Límite de tiempo: 00:15 Hora/Minutos
Método de navegación: Secuencial
Tipo de evaluación: Formativa (autoevaluación)
Método de calificación: Calificación más alta

Resumen de sus intentos previos

Intento	Estado	Calificación obtenida	Calificación final / 10.00	Revisión
1	Finalizado Martes, 11 de Agosto de 2026, 12:53	17.45 / 20.00	8.73	Revisión

Calificación más alta: 8.73 / 10.00

Revisión de intento
INGENIERÍA DE REQUERIMIENTOS - SOFT-R - A - (4TO NIVEL) | CORTE 2

Estudiante	MESIAS QUIJJE JHON ALEXANDER
Comenzado el	Martes, 11 de Agosto de 2026, 12:40
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 11 de Agosto de 2026, 12:53
Tiempo empleado	0:13:08.005924
Calificación obtenida	17.45 / 20.00
Calificación final	8.73 / 10.00

NAVEGACIÓN

1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20				

Correcto

Parcial

Incorrecto

Índice

1	Caso práctico	2
2	P1. Modelo de datos – Diagrama de clases UML	2
3	P2. Modelo funcional – Diagrama de actividades UML	2
4	P3. Modelo de comportamiento – Máquina de estados UML	3
5	P4. Consistencia entre las tres perspectivas	4
6	P5. Especificación de requisitos con esquema de atributos	5
7	P6. Priorización MoSCoW	5
8	P7. Validación por inspección (checklist ISO/IEC/IEEE 29148)	6
9	P8. Pruebas de aceptación trazadas	7
10	P9. Matriz de trazabilidad	8
11	P10. Gestión del cambio y línea base	8

1. Caso práctico

Sistema de Gestión de Pedidos de una tienda en línea: un *Cliente* realiza cero o más *Pedidos*; cada *Pedido* contiene una o más *Líneas de pedido*, cada una referenciando un *Producto*. Al registrar un pedido se verifica el stock: si hay existencias se confirma, si no se notifica el faltante. Un pedido confirmado puede despacharse, luego entregarse, o anularse mientras no haya sido entregado.

2. P1. Modelo de datos – Diagrama de clases UML

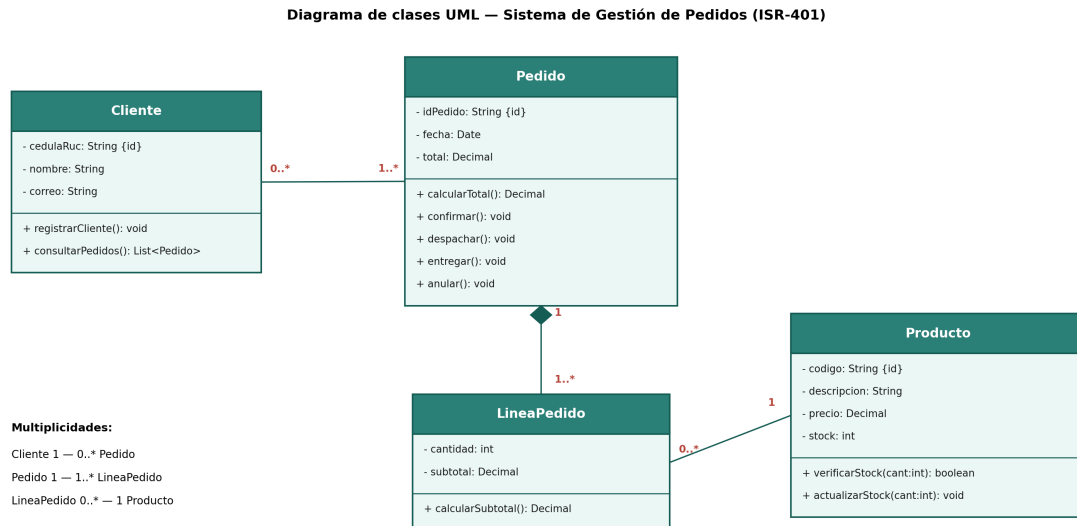


Figura 1: Diagrama de clases del dominio Sistema de Gestión de Pedidos

Los identificadores marcados con {id} son `Cliente.cedulaRuc`, `Pedido.idPedido` y `Producto.codigo`; `LineaPedido` se identifica por su posición dentro del `Pedido`, reforzada por la relación de **composición** (rombo relleno) entre `Pedido` y `LineaPedido`: una línea no existe sin su pedido y se elimina junto con él. Las multiplicidades declaradas son: `Pedido 1 — 1..* LineaPedido`, y `LineaPedido 0..* — 1 Producto` (cada línea referencia exactamente un producto, y un producto puede aparecer en cero o más líneas).

3. P2. Modelo funcional – Diagrama de actividades UML

El proceso inicia en el carril *Cliente* con el nodo inicial, seguido de la acción *Enviar pedido (con líneas de productos)*, que cruza al carril *Sistema*. Allí el flujo recibe el pedido y procesa cada línea (*Por cada línea del pedido*) hasta llegar al nodo de decisión *¿Hay stock suficiente?*. La rama *Sí* confirma la línea y actualiza el stock del producto (descuenta la cantidad); la rama *No* notifica el faltante. Ambos caminos convergen en un nodo de unión antes de *Calcular total del pedido y Confirmar pedido*, cerrando en el nodo final del carril *Sistema*.

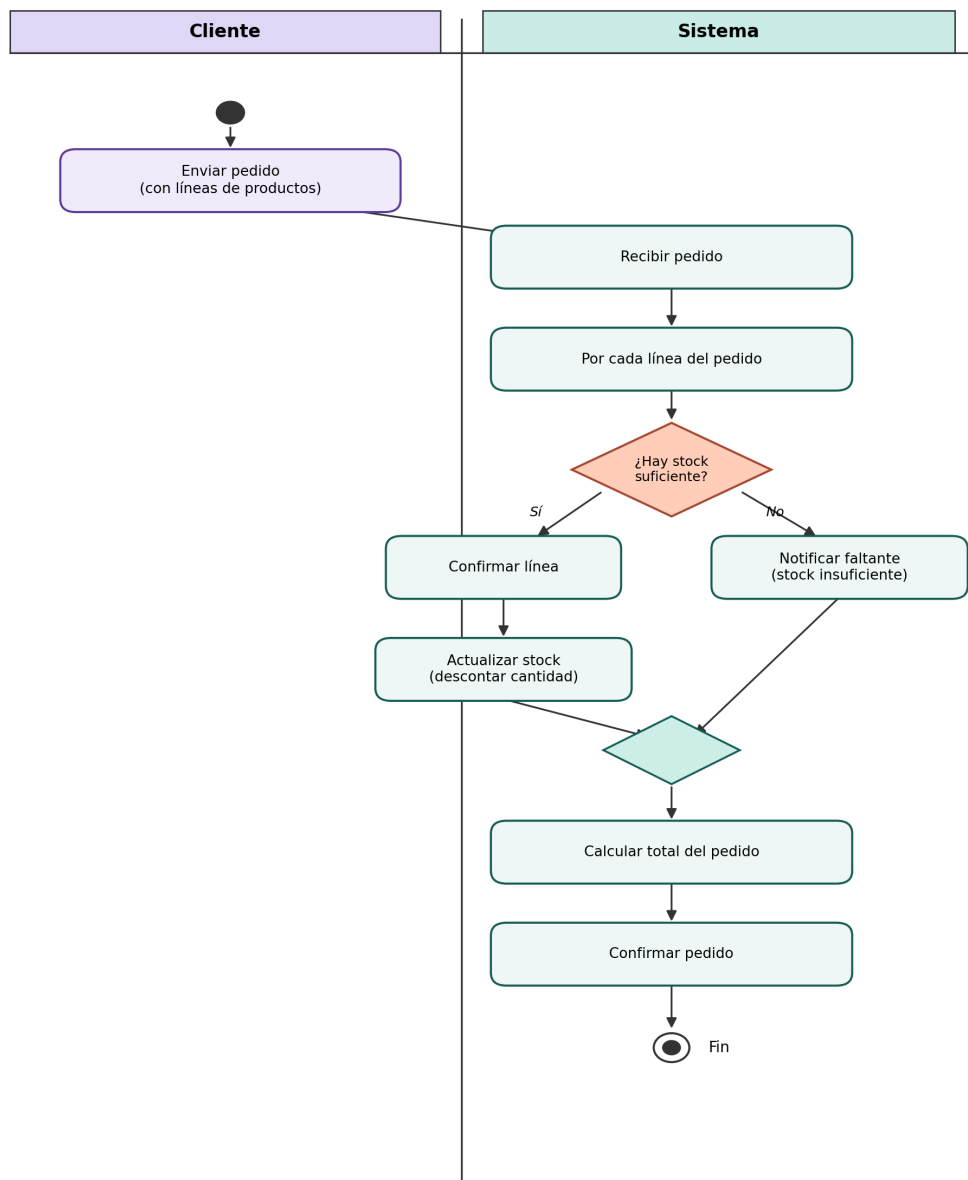
P2. Diagrama de actividades — "Registrar pedido"

Figura 2: Diagrama de actividades del proceso "Registrar pedido", con carriles Cliente / Sistema

4. P3. Modelo de comportamiento – Máquina de estados UML

El superestado **Activo** agrupa los estados *Confirmado* y *Enviado*, factorizando en una sola transición de salida el evento `anular()` [`not entregado`] hacia *Cancelado* (estado final). El estado *Registrado* es el estado inicial y tiene una transición reflexiva `notificarFaltante()` [`stock = 0`] que lo mantiene en *Registrado* cuando la verificación de stock falla. El evento `confirmar()` [`stock disponible`] lleva de *Registrado* a *Confirmado*; `despachar()` lleva a *Enviado*; `entregar()` lleva a *Entregado*, marcado como estado final de aceptación.

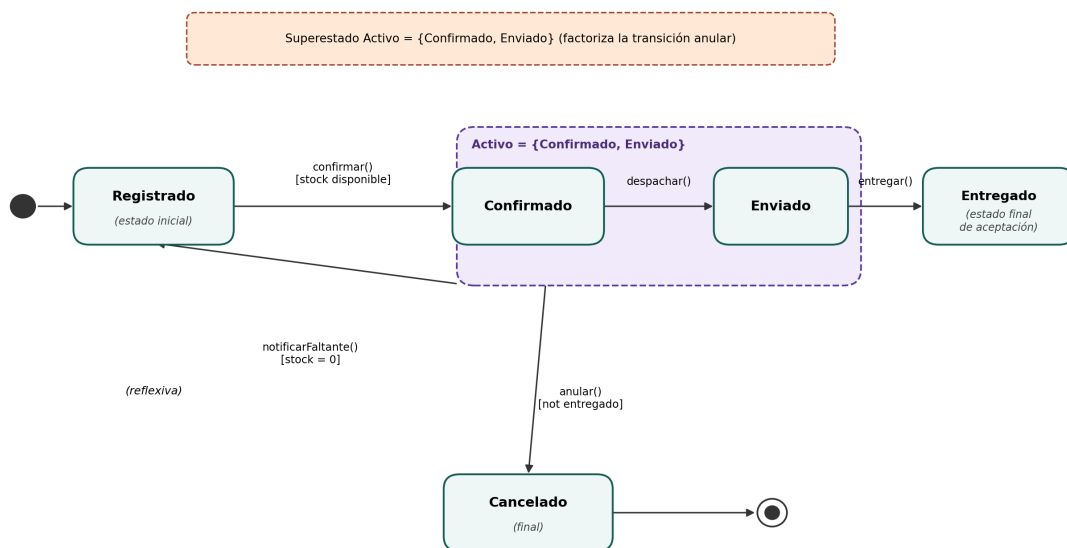
P3. Máquina de estados del Pedido

Figura 3: Máquina de estados del ciclo de vida de un Pedido

5. P4. Consistencia entre las tres perspectivas**Tabla de verificación cruzada**

Evento (P3)	Función/acción (P2)	Entidad/atributo (P1) modificado
confirmar() [stock disponible]	Confirmar línea → Actualizar stock (rama Sí)	Producto.stock vía actualizarStock(cant); estado del Pedido
notificarFaltante() [stock = 0]	Notificar faltante (stock insuficiente) (rama No)	(sin operación equivalente declarada en ninguna clase de P1)
despachar()	(sin acción explícita en P2; P2 modela solo Registrar pedido)	Pedido.despachar()
entregar()	(sin acción explícita en P2)	Pedido.entregar()
anular() [not entregado]	(sin acción explícita en P2)	Pedido.anular()

Inconsistencia detectada

Defecto (P1, asociación Cliente–Pedido): en el diagrama de clases, las multiplicidades dibujadas junto a la línea de asociación son 0..* en el extremo de Cliente y 1..* en el extremo de Pedido. Esto contradice tanto la leyenda del propio diagrama (Cliente 1 – 0..* Pedido) como la regla de negocio del caso (un Cliente [...] puede realizar cero o más Pedidos): con la lectura dibujada, un Pedido quedaría asociado a 0..* Clientes (un pedido sin dueño claro) y cada Cliente exigiría 1..* Pedidos (ningún cliente podría tener cero pedidos, lo que rompe el enunciado del caso).

Corrección aplicada: se invierten las multiplicidades del extremo de la asociación Cliente– Pedido para que queden 1 junto a Cliente y 0..* junto a Pedido, alineando el diagrama con su propia leyenda y con la regla de negocio del caso. (Ajuste recomendado sobre el archivo fuente del diagrama de clases antes de la entrega final.)

Defecto secundario (P1 vs. P3): el evento `notificarFaltante()` (transición reflexiva sobre Registrado en P3, y acción Notificar faltante en P2) no tiene una operación equivalente en ninguna clase de P1 – Pedido solo declara `calcularTotal()`, `confirmar()`, `despachar()`, `entregar()` y `anular()`. Se recomienda añadir `+ notificarFaltante(): void` a la clase Pedido para cerrar la trazabilidad P1–P2–P3.

6. P5. Especificación de requisitos con esquema de atributos

- **RF-01:** El sistema debe registrar un nuevo pedido validando la disponibilidad de stock de cada producto solicitado.
- **RF-02:** El sistema debe permitir anular un pedido siempre que no haya sido entregado.
- **RNF-01** (Eficiencia de desempeño – ISO/IEC 25010): El sistema debe confirmar o rechazar el registro de un pedido en menos de 3 segundos bajo carga normal (≤ 50 pedidos concurrentes).
- **RNF-02** (Seguridad – ISO/IEC 25010): Los datos personales del cliente (cédula/RUC, correo) deben almacenarse cifrados en reposo (AES-256) y transmitirse únicamente mediante TLS 1.2 o superior.

ID	Prioridad	Estado	Fuente	Estabilidad	Ver.	Método de verificación
RF-01	Alta	Propuesto	Caso de estudio, 2 (regla de stock)	Estable	1.0	Prueba funcional (CP-01)
RF-02	Alta	Propuesto	Caso de estudio, 2 (ciclo de vida del pedido)	Estable	1.0	Prueba funcional (CP-02)
RNF-01	Media	Propuesto	Requisito de negocio (experiencia de usuario)	Media	1.0	Prueba de rendimiento/carga (CP-03)
RNF-02	Alta	Propuesto	Política de protección de datos personales	Estable	1.0	Prueba de seguridad / auditoría de cifrado (CP-04)

7. P6. Priorización MoSCoW

- **RF-01 – Must have.** Es la funcionalidad núcleo del dominio: sin verificación y registro de pedidos el sistema no cumple su propósito.
- **RNF-02 – Must have.** La protección de datos personales es una obligación explícita del caso (proteger los datos personales del cliente) y de cumplimiento normativo; omitirla implica riesgo legal y de negocio.
- **RF-02 – Should have.** Importante para completar el ciclo de vida del pedido, pero el sistema puede operar en un primer incremento sin anulación automatizada (proceso manual temporal).

- **RNF-01 – Could have.** Deseable para una buena experiencia de usuario, pero no bloquea la operación básica del sistema si se entrega en una iteración posterior.

8. P7. Validación por inspección (checklist ISO/IEC/IEEE 29148)

Paso 1 – Identificación de defectos (sin corregir)

Propiedades evaluadas: no ambiguo, completo, singular, verificable, factible.

ID	Requisito	Propiedad violada	Descripción del defecto
D-01	RF-01	Completo	No define el comportamiento cuando solo una parte de los productos del pedido tiene stock disponible.
D-02	RF-02	No ambiguo	No especifica qué rol/actor está autorizado para anular el pedido.
D-03	RNF-01	Verificable	No define el entorno/condiciones de medición (hardware, red) necesarias para reproducir la métrica de 3 s.
D-04	RNF-02	Singular	Combina dos mecanismos de seguridad distintos (cifrado en reposo y cifrado en tránsito) en un mismo enunciado.

Paso 2 – Retrabajo (requisitos corregidos)

- **RF-01 (v1.1):** El sistema debe registrar un pedido solo si todos los productos solicitados tienen stock suficiente; si algún producto no tiene stock suficiente, el sistema debe rechazar el registro e indicar el/los producto(s) que generaron el faltante.
- **RF-02 (v1.1):** El sistema debe permitir que el Cliente propietario del pedido o un Administrador anulen un pedido, siempre que su estado no sea Entregado.
- **RNF-01 (v1.1):** En un entorno de referencia (servidor de 4 vCPU / 8 GB RAM, red interna ≤ 5 ms de latencia) y bajo carga normal (≤ 50 pedidos concurrentes), el sistema debe confirmar o rechazar el registro de un pedido en menos de 3 segundos en el 95 % de las solicitudes.
- **RNF-02a (v1.1):** Los datos personales del cliente (cédula/RUC, correo) deben almacenarse cifrados en reposo utilizando AES-256.
- **RNF-02b (v1.1):** Toda comunicación cliente-sistema que transmita datos personales debe utilizar TLS 1.2 o superior.

9. P8. Pruebas de aceptación trazadas

ID	Tipo	Entradas	Criterio de éxito	Traza
CP-01	Funcional	Pedido con producto A (stock=5, solicita 2) y producto B (stock=0, solicita 1)	El sistema rechaza el pedido completo e indica el código del producto B como causante del faltante	RF-01
CP-02	Funcional	Pedido en estado Confirmado; solicitud de anulación por el cliente propietario; caso negativo: solicitud sobre pedido Entregado	El pedido pasa a Cancelado con fecha/hora registrada; el caso negativo es rechazado	RF-02
CP-03	No funcional (rendimiento)	50 solicitudes de registro de pedido corrientes en el entorno de referencia	El percentil 95 del tiempo de respuesta es menor a 3 s	RNF-01
CP-04	No funcional (seguridad) / regresión	Registro de un cliente nuevo; inspección de base de datos y captura de tráfico de red durante el registro	El campo correo/cédula aparece cifrado en la base de datos y el tráfico usa TLS 1.2+	RNF-02a, RNF-02b

10. P9. Matriz de trazabilidad

Req.	Fuente (traza atrás)	Modelos (traza adelante)	Prueba	Observación
RF-01	Caso de estudio 2: el sistema verifica la disponibilidad de stock	P1: <code>Producto.stock</code> , <code>verificarStock()</code> , <code>actualizarStock()</code> ; P2: ¿Hay stock suficiente? / Actualizar stock; P3: <code>confirmar()</code> [<code>stock disponible</code>], <code>notificarFaltante()</code> [<code>stock=0</code>]	CP-01	–
RF-02	Caso de estudio 2: puede [...] anularse mientras no haya sido entregado	P3: <code>anular()</code> [not entregado] (transición del superestado Activo); P1: <code>Pedido.anular()</code>	CP-02	Depende de RF-01 (dependencia horizontal: no puede anularse un pedido que no ha sido registrado)
RNF-01	Requisito de negocio (registrar pedidos con rapidez)	Sin modelo estructural/comportamental directo en P1/P2/P3	CP-03	Huérfano de modelo: las NFR de rendimiento no se representan en los diagramas UML estructurales; solo tiene traza a prueba
RNF-02	Caso de estudio 2: proteger los datos personales del cliente	P1: atributos <code>Cliente.cedulaRuc</code> , <code>Cliente.correo</code>	CP-04	–

Dependencia horizontal: RF-02 → RF-01 (un pedido debe existir/registrarse antes de poder anularse).

Requisito huérfano: RNF-01 carece de un elemento de modelo (P1/P2/P3) que lo represente explícitamente; solo se verifica mediante prueba de carga (CP-03). Se recomienda documentarlo como una nota de rendimiento asociada a las operaciones `Pedido.confirmar()` y `Producto.verificarStock()`.

11. P10. Gestión del cambio y línea base

Solicitud de cambio

SC-01 – Solicitante: Jefe de Ventas. Fecha: 2026-08-11. Requisito afectado: RF-02 (v1.1). Descripción: permitir que, además del Cliente propietario y el Administrador, un Supervisor de bodega pueda anular un pedido cuando detecte un error de inventario.

Análisis de impacto (usando la matriz de P9)

- Afecta directamente a RF-02 y a su prueba CP-02 (debe añadirse un caso con actor Supervisor de bodega).

- Por la dependencia horizontal RF-02 → RF-01, RF-01 no se ve afectado.
- Si se decide registrar quién ejecuta la anulación, debe agregarse el atributo `Pedido. actorAnulacion: String` en P1 (impacto menor en el diagrama de clases). La máquina de estados (P3) no cambia: la guarda [pedido no entregado] sigue siendo válida para el nuevo actor.

Decisión del CCB

Aprobada (Comité de Control de Cambios, 2026-08-13), condicionada a: (1) agregar `Pedido. actorAnulacion` en P1, y (2) actualizar CP-02 con el nuevo caso de prueba para el Supervisor de bodega. Nueva versión de RF-02: v1.2.

Línea base

Baseline B1.0 (2026-08-11, antes del cambio): ERS v1.1 (requisitos reworked de P7), P1 v1.0, P2 v1.0, P3 v1.0, matriz de trazabilidad (P9) v1.0, pruebas de aceptación (P8) v1.0.

Baseline B1.1 (2026-08-13, tras aprobación de SC-01): RF-02 v1.2, P1 v1.1 (con `actorAnulacion`), CP-02 v1.1, matriz de trazabilidad v1.1; el resto de artefactos hereda sin cambios de B1.0.